

Introducción a Funciones, Procedimientos y Triggers en PostgreSQL

Funciones en PostgreSQL

Las funciones en PostgreSQL son bloques de código reutilizables que realizan una tarea específica. Pueden aceptar parámetros de entrada, procesar datos y devolver un resultado. Son una herramienta esencial para la modularidad, la eficiencia y la legibilidad del código SQL.





Tipos de funciones

1 **Funciones escalares**

Las funciones escalares devuelven un único valor. Son las más comunes y se utilizan para realizar operaciones como cálculos, conversiones de datos y validaciones.

2 **Funciones SETOF**

Estas funciones devuelven un conjunto de valores. Se utilizan para realizar operaciones que producen múltiples resultados, como la selección de datos de una tabla.

3 **Funciones de procedimiento**

Las funciones de procedimiento no devuelven un valor, sino que realizan acciones como la actualización de datos o la ejecución de otros comandos SQL.

4 **Funciones agregadas**

Las funciones agregadas operan sobre conjuntos de datos y devuelven un único valor que resume los datos del conjunto. Ejemplos de funciones agregadas son SUM(), AVG() y COUNT().

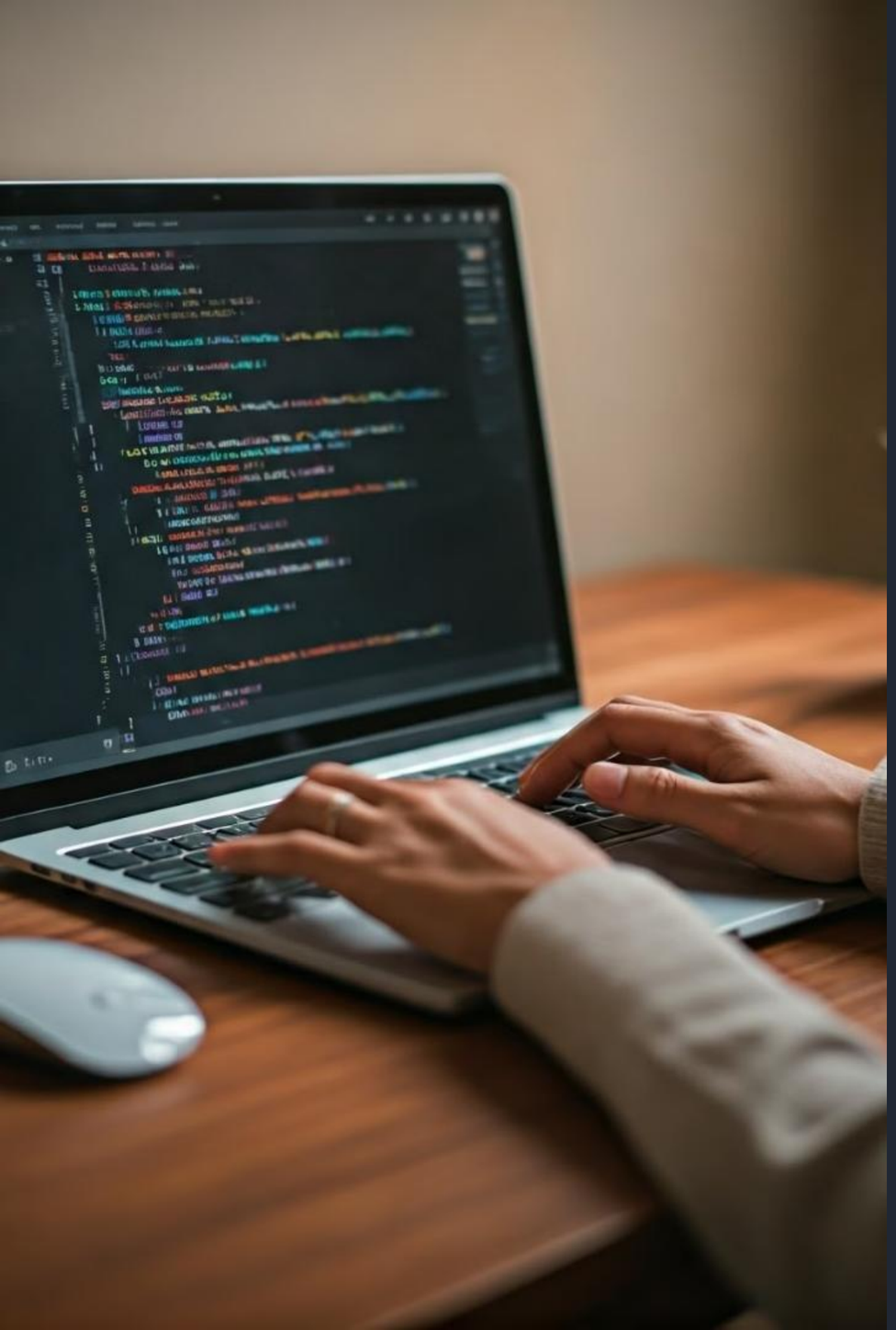


Definición de funciones

Las funciones se definen usando la sentencia CREATE FUNCTION. La sintaxis general es la siguiente:

```
CREATE OR REPLACE FUNCTION nombre_funcion(parametro1
tipo1, parametro2 tipo2, ...)
RETURNS tipo_retorno
LANGUAGE plpgsql
AS $$
DECLARE
    -- declaración de variables locales (opcional)
BEGIN
    -- cuerpo de la función
    RETURN valor;
END;
$$;
```

En el código de la función, se utiliza la sentencia RETURN para devolver el resultado de la función.



Uso de funciones

Las funciones se pueden usar en consultas SQL como si fueran cualquier otra columna o expresión. Por ejemplo, para calcular el IVA de un producto con un precio de 100.000p esos, se puede utilizar la función calcular_iva() de la siguiente manera:

```
SELECT calcular_iva(100000);
```

Esto devolverá el valor 21, que es el IVA del producto.

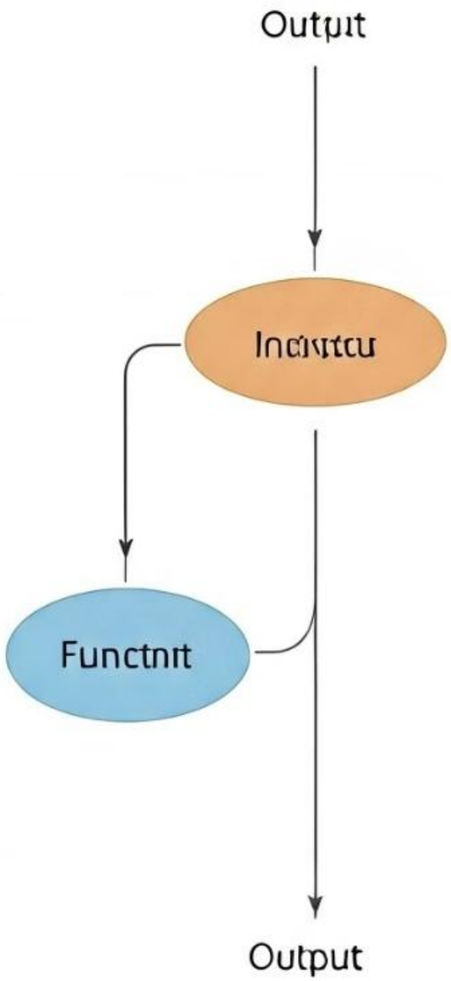


Funciones con parámetros de entrada

Las funciones pueden aceptar parámetros de entrada que se utilizan para personalizar el comportamiento de la función. Los parámetros se declaran entre paréntesis después del nombre de la función, separados por comas. Cada parámetro tiene un nombre y un tipo de dato.

```
CREATE FUNCTION calcular_descuento(precio DECIMAL, porcentaje DECIMAL)
RETURNS DECIMAL
AS $$
BEGIN
    RETURN precio * (1 - porcentaje);
END;
$$ LANGUAGE plpgsql;
```

Esta función calcula el descuento de un precio, tomando como entrada el precio original y el porcentaje de descuento.



Funciones con valores de retorno

Las funciones devuelven un valor utilizando la sentencia RETURN. El tipo de dato del valor devuelto debe coincidir con el tipo de dato declarado en la cláusula RETURNS de la definición de la función. Las funciones pueden devolver cualquier tipo de dato, incluidos tipos de datos complejos como registros y arreglos.

```
CREATE FUNCTION obtener_nombre_completo(nombre TEXT, apellido TEXT)
RETURNS TEXT
AS $$
BEGIN
    RETURN nombre || ' ' || apellido;
END;
$$ LANGUAGE plpgsql;
```

Esta función concatena el nombre y el apellido para obtener el nombre completo de una persona.



Beneficios del uso de funciones

Reutilización de código	Las funciones permiten reutilizar bloques de código, evitando la repetición de código y mejorando la modularidad.
Eficiencia	Las funciones pueden mejorar la eficiencia del código al permitir que el servidor de base de datos ejecute el código una sola vez y almacenar los resultados en caché para uso futuro.
Legibilidad	Las funciones hacen que el código sea más legible y fácil de mantener al dividir el código en componentes más pequeños y manejables.
Seguridad	Las funciones pueden ayudar a mejorar la seguridad al encapsular la lógica de negocio y controlar el acceso a los datos.

¿Qué son los triggers?

Los triggers son objetos que ejecutan una función especial al detectar un evento. Su objetivo es realizar acciones adicionales en respuesta a estos eventos, asegurando la integridad de la base de datos.

1 Automatización

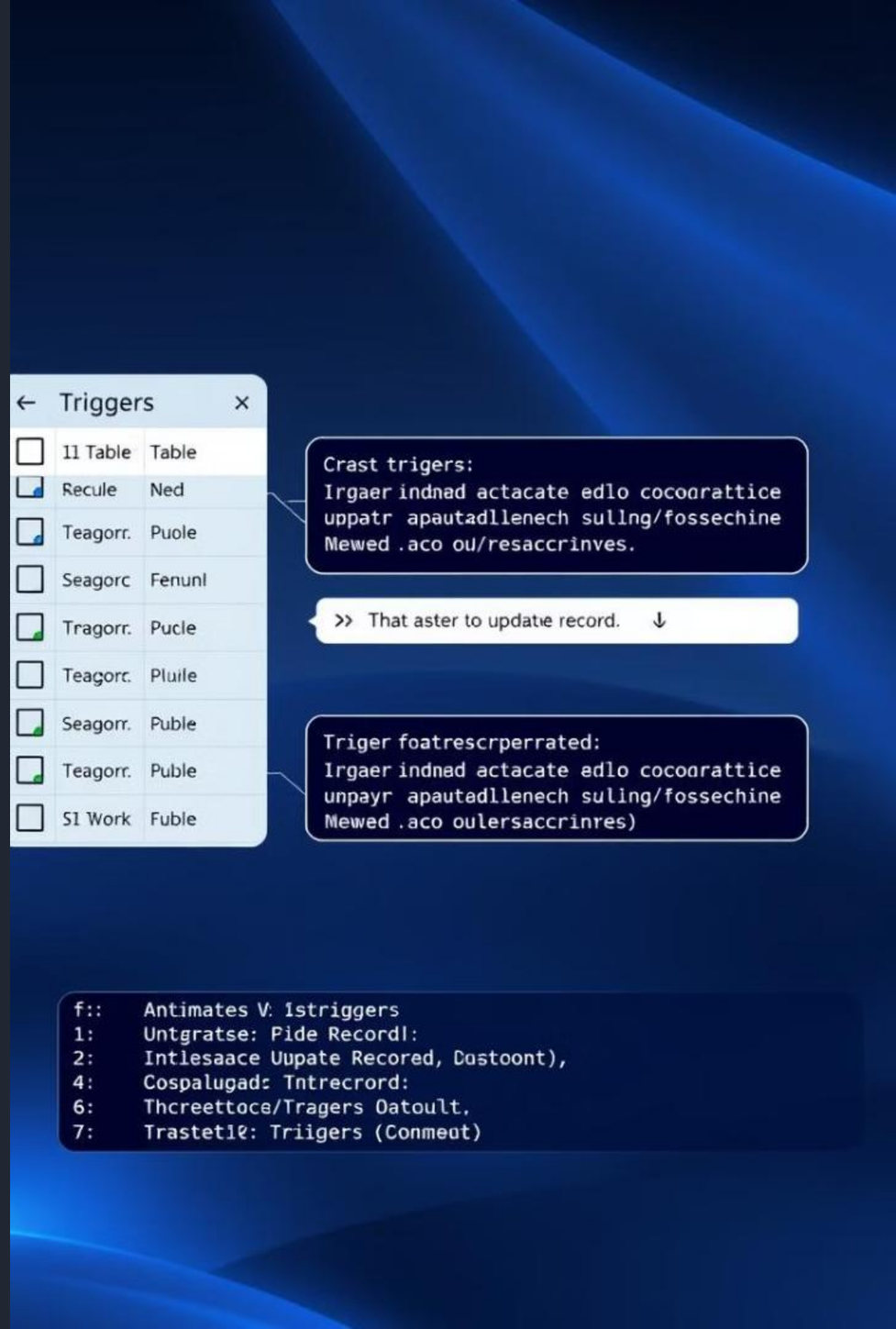
Los triggers permiten la automatización de tareas relacionadas con la manipulación de datos.

2 Integridad

Se pueden usar para validar datos, garantizar restricciones y mantener la coherencia de la información.

3 Auditoría

Los triggers pueden registrar los cambios en la base de datos, permitiendo la trazabilidad de las modificaciones.





Características de los triggers

Se ejecutan automáticamente cuando ocurre un evento específico en la base de datos, como una inserción, actualización o eliminación de datos.

1 Eventos

Los triggers se asocian a eventos específicos como INSERT, UPDATE o DELETE, que ocurren en una tabla específica.

2 Tiempo de Activación

Los triggers pueden activarse antes o después de que ocurra el evento asociado.

3 Acciones

Al activarse, los triggers pueden realizar acciones como actualizar datos, insertar nuevos registros o generar mensajes.



Modelo ECA(Event-Condition-Action)

EVENTO

Acción que provoca la activación del trigger (por ejemplo: INSERT, UPDATE, DELETE).

CONDICION

Es una evaluación lógica opcional (WHEN) que determina si la acción debe llevarse a cabo. Si la condición se evalúa como verdadera, el trigger prosigue; si no se cumple, la acción no se ejecuta, lo que optimiza el rendimiento.

ACCION

El bloque de código, sentencia SQL o procedimiento que se ejecuta de forma automática en respuesta al evento.



Creación y activación de triggers

Para crear un trigger, se utiliza la sentencia CREATE TRIGGER, especificando el nombre, la tabla, el evento que lo activa y la función que se ejecutará.

1

Definir Trigger

Se define el nombre, la tabla, el evento y la función asociada.

2

Asociar Función

Se especifica la función que se ejecutará cuando se produzca el evento.

3

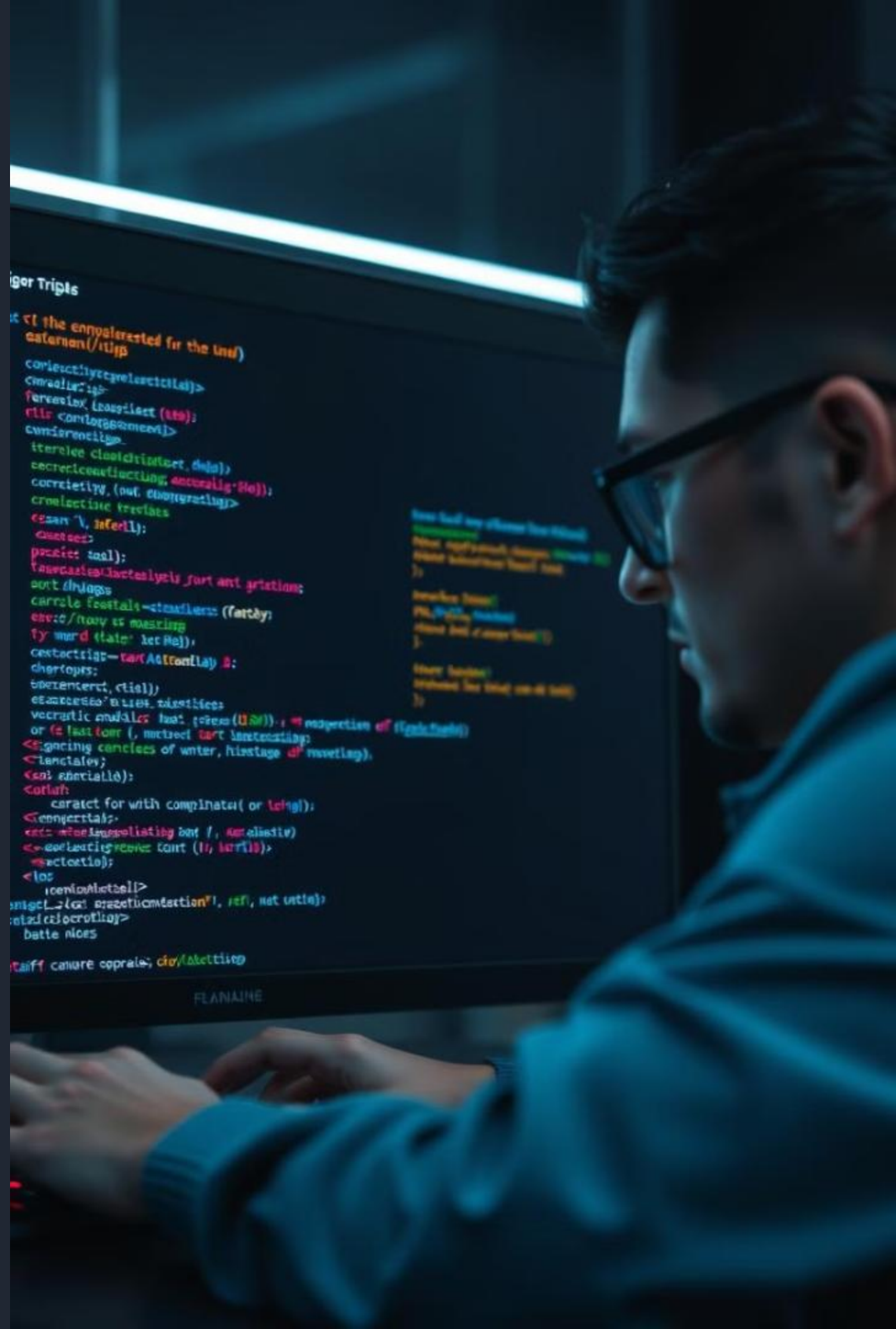
Activar Trigger

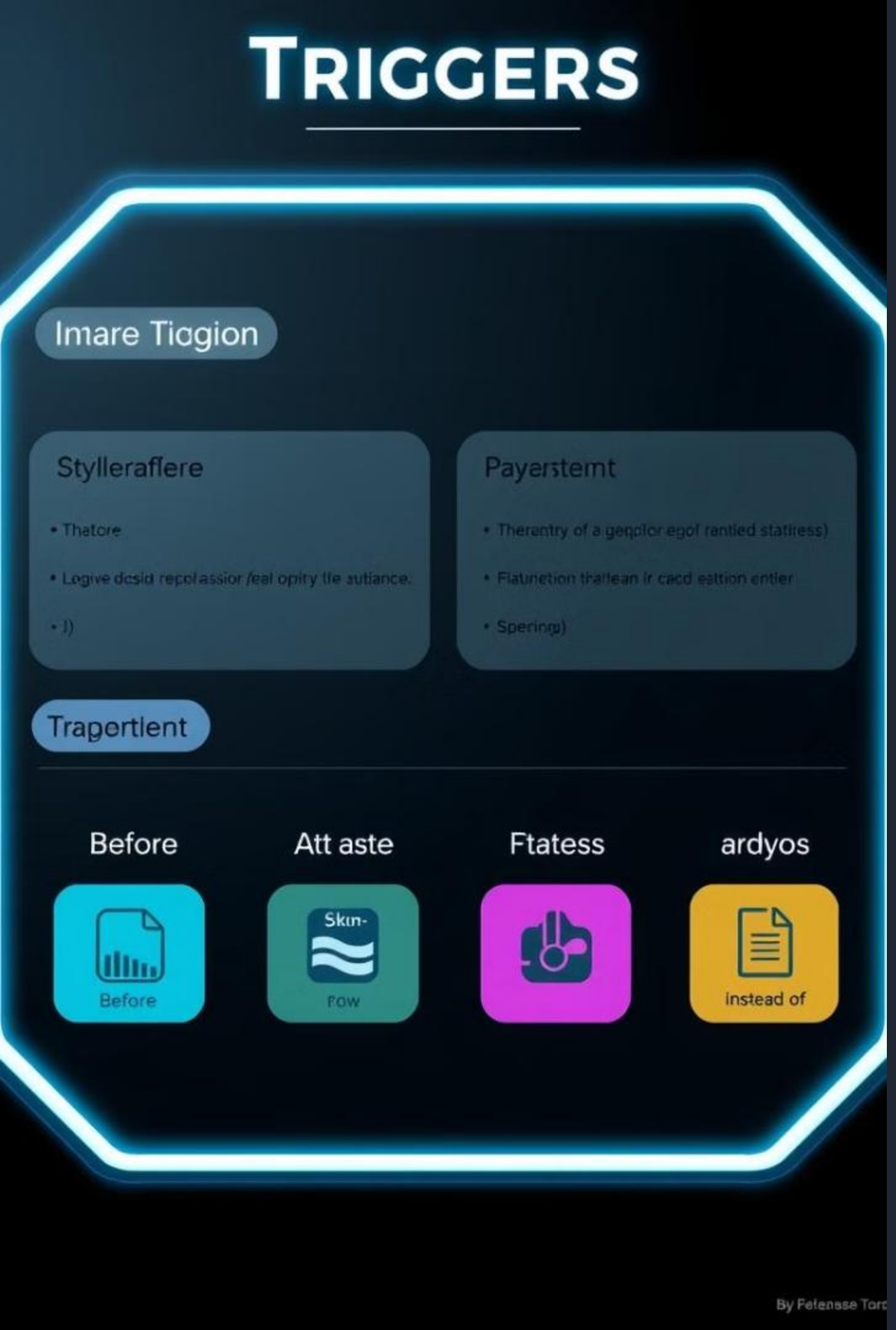
El trigger se activa automáticamente al producirse el evento en la tabla.

Creación de triggers

La creación de triggers implica definir el nombre del trigger, la tabla, el evento asociado, el tiempo de activación y el código SQL que se ejecutará cuando se active el trigger.

```
CREATE TRIGGER valida_sueldo
BEFORE INSERT ON EMPLEADO
FOR EACH ROW
EXCECUTE FUNCTION valida_sueldo();
```





Tipos de triggers en PostgreSQL

Los triggers en PostgreSQL se pueden clasificar según el momento de su ejecución (antes o después del evento) y el nivel de granularidad (fila o sentencia).

Tipo	Descripción
BEFORE	Se ejecutan antes de que se produzca el evento en la tabla.
AFTER	Se ejecutan después de que se produzca el evento en la tabla.
ROW	Se ejecutan para cada fila afectada por el evento.
STATEMENT	Se ejecutan una vez por cada sentencia que produce el evento.
INSTEAD OF	Se ejecutan en lugar de la operación original.

¿Qué son los procedimientos?

Los procedimientos almacenados en PostgreSQL son bloques de código que encapsulan una serie de instrucciones SQL. Estos procedimientos se ejecutan como una unidad y pueden aceptar parámetros de entrada, manipular datos y devolver resultados, simplificando operaciones complejas.

Reusabilidad

Los procedimientos se pueden reutilizar en diferentes partes de la aplicación, ahorrando código y evitando la duplicación.

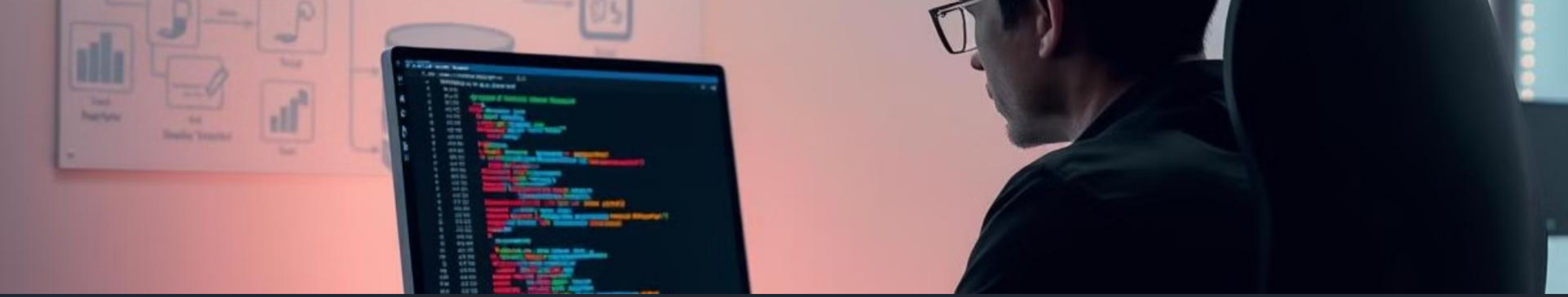
Seguridad

Los procedimientos pueden limitar el acceso a los datos, proporcionando un nivel adicional de seguridad.

Optimización

El motor de la base de datos puede optimizar la ejecución de los procedimientos, mejorando el rendimiento.





Definición de procedimientos

Se ejecutan como una unidad y se pueden llamar varias veces desde diferentes puntos de nuestra aplicación.

Beneficios

Simplifican el código de la aplicación, mejoran el rendimiento, permiten la reutilización del código, y aumentan la seguridad al centralizar las operaciones en la base de datos.

Sintaxis

Los procedimientos se declaran con la palabra clave CREATE PROCEDURE, seguido del nombre del procedimiento, los parámetros y el bloque de código SQL a ejecutar.

Creación y ejecución de procedimientos

Crear un procedimiento en PostgreSQL es sencillo. Se utiliza la sentencia CREATE PROCEDURE, definiendo el nombre del procedimiento, los parámetros de entrada y el cuerpo del procedimiento.

1

Definir Procedimiento

Se utiliza la sentencia CREATE PROCEDURE con el nombre, parámetros y cuerpo.

2

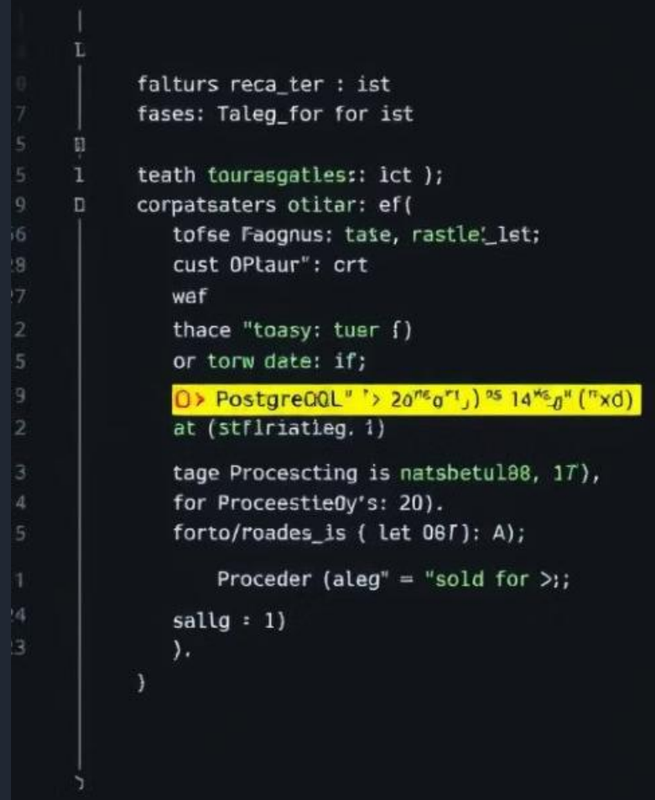
Asignar Permisos

Es posible que se necesiten permisos para ejecutar el procedimiento, especialmente en entornos con control de acceso.

3

Ejecutar Procedimiento

Se invoca el procedimiento utilizando su nombre y los parámetros de entrada.



Creación de procedimientos

La creación de procedimientos es un proceso sencillo que implica definir el nombre, los parámetros y el código SQL que se ejecutará dentro del procedimiento. Es importante probar cuidadosamente nuestros procedimientos para garantizar su correcto funcionamiento.

Ejemplo

```
CREATE OR REPLACE PROCEDURE actualizar_precio(id_producto INTEGER,
nuevo_precio NUMERIC)
AS $$
BEGIN
 UPDATE productos SET precio = nuevo_precio WHERE id =id_producto;
END;
$$
LANGUAGE plpgsql;
```

Explicación

El procedimiento actualizar_precio toma dos parámetros, el ID del producto y el nuevo precio. Actualiza la tabla productos con el nuevo precio del producto especificado.

Ejecución de procedimientos

Los procedimientos se ejecutan llamando su nombre y proporcionando los parámetros necesarios. La ejecución de un procedimiento puede resultar en la actualización de datos, la inserción de nuevos registros o la realización de otras operaciones en la base de datos.

1

Llamada

CALL actualizar_precio(1, 10.99);

2

Ejecución

El procedimiento se ejecuta, actualiza la tabla productos y devuelve el resultado de la operación.



Casos de uso y beneficios de los procedimientos, triggers y funciones

Los procedimientos, triggers y funciones ofrecen una variedad de aplicaciones en el desarrollo de bases de datos, mejorando la eficiencia, la seguridad y la integridad de los datos.

Seguridad

Los procedimientos pueden restringir el acceso a datos sensibles.

Automatización

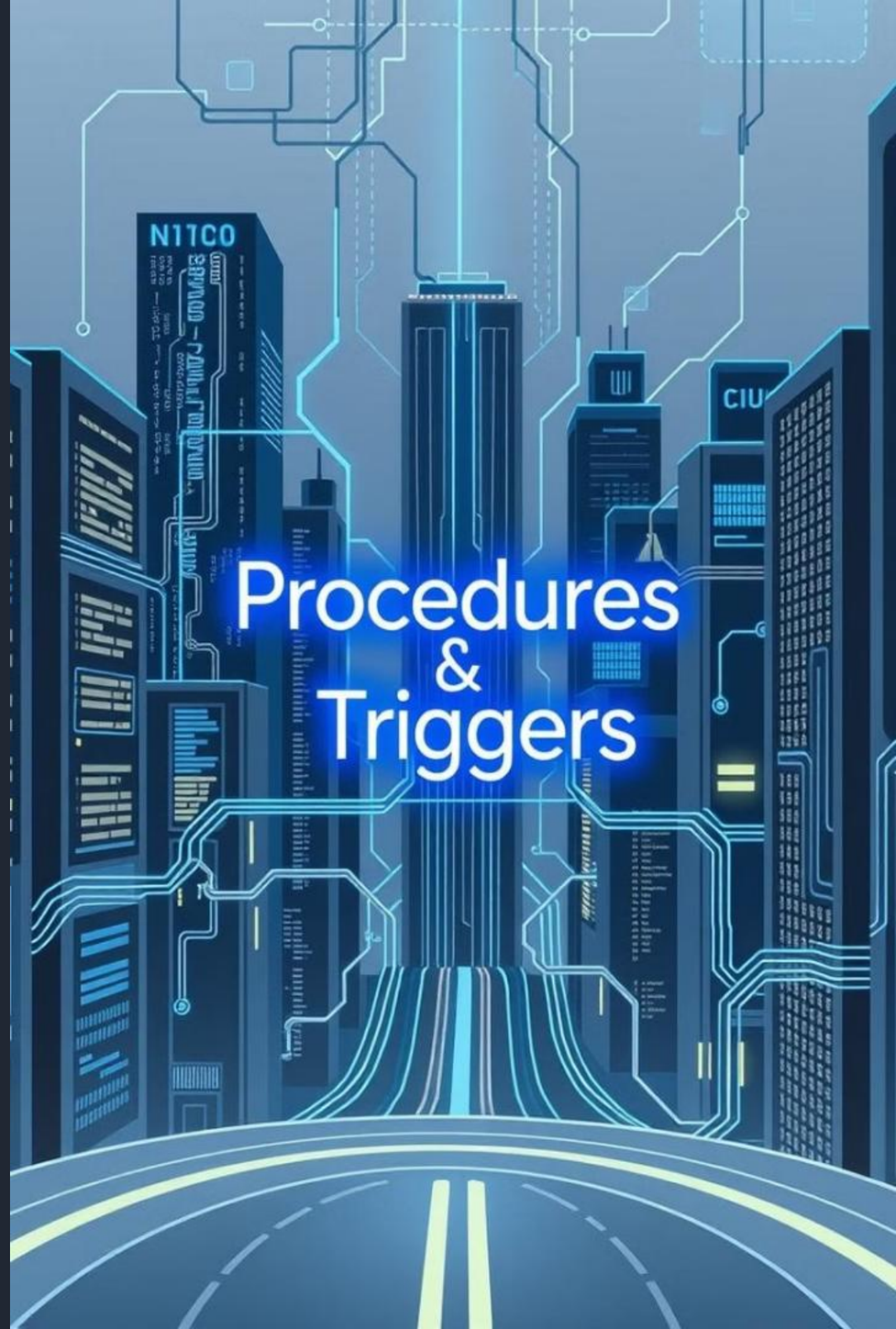
Los triggers simplifican tareas repetitivas.

Mantenimiento

Se facilita la aplicación de reglas y restricciones.

Flexibilidad

Los triggers permiten la personalización de la lógica de la base de datos.



Ejemplo de Funciones.

Supongamos que tenés una tabla de empleados y querés calcular el salario anual de cada uno (asumiendo que el salario se ingresa de manera mensual)

```
CREATE OR REPLACE FUNCTION salario_anual(salario_mensual
numeric)
RETURNS numeric
AS $$
BEGIN
    RETURN salario_mensual * 12;
END;
$$
LANGUAGE plpgsql;

Luego:
SELECT nombre, salario_anual(salario_mensual) AS
salario_anual FROM empleados;
```



Ejemplo de Procedimiento.

Crea un procedimiento que asigne un empleado a un proyecto y defina su rol en dicho proyecto. Utiliza este procedimiento para asignar al empleado con empleado_id = 1 al proyecto con proyecto_id = 2 con el rol de "Líder de proyecto".

```
CREATE OR REPLACE PROCEDURE asignar_empleado_proyecto(
p_empleado_id INT,
p_proyecto_id INT,
p_rol VARCHAR(50)
)
LANGUAGE plpgsql
AS $$
BEGIN
INSERT INTO proyectos.empleados_proyectos(empleado_id,proyecto_id,
rol) VALUES (p_empleado_id, p_proyecto_id, p_rol);
RAISE NOTICE 'Empleado % asignado al proyecto % con el rol %',
p_empleado_id, p_proyecto_id, p_rol;
END;
$$;
```



Ejemplo de Triggers.

Supongamos que queremos actualizar la tabla de empleados cuando un es actualizado y que además guarde el último salario anterior.

1) Necesitamos crear una función que será ejecutada por el trigger. Esta función, definirá qué acción tomar cuando se actualice el salario.

```
CREATE OR REPLACE FUNCTION registrar_cambio_salario()
RETURNS TRIGGER AS $$
BEGIN
    IF NEW.salario <> OLD.salario THEN
        INSERT INTO historial_salarios(empleado_id, salario_anterior, salario_nuevo)
        VALUES (OLD.id, OLD.salario, NEW.salario);
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```



Ejemplo de Triggers.

Finalmente, creamos el trigger que ejecutará la función cada vez que se actualice la tabla de empleados

```
CREATE TRIGGER trigger_salario_actualizado
AFTER UPDATE ON empleados
FOR EACH ROW
WHEN (OLD.salario IS DISTINCT FROM NEW.salario)
EXECUTE FUNCTION registrar_cambio_salario();
```

